1.Decentralized Voting System using Solidity in Remix

Step 1: Open Remix IDE

Open the official website:

Remix IDE

Step 2: Create a New Solidity File

VotingSystem.sol

Step 3: Write the Smart Contract Code

Paste the following code inside the file:

```
// SPDX-License-Identifier: MIT

pragma solidity ^0.8.0;


contract VotingSystem {


    address public admin;

    bool public votingStarted;

    bool public votingEnded;


    struct Candidate {

        uint id;
```

```
    string name;
    uint voteCount;
}

mapping(uint => Candidate) public candidates;
mapping(address => bool) public hasVoted;

uint public candidatesCount;

constructor() {
    admin = msg.sender;

    addCandidate("Alice");
    addCandidate("Bob");
    addCandidate("Charlie");
}

modifier onlyAdmin() {
    require(msg.sender == admin, "Only admin can perform this action");
    _;
}

function addCandidate(string memory _name) private {
    candidatesCount++;
```

```
    candidates[candidatesCount] = Candidate(candidatesCount, _name, 0);
}

function startVoting() public onlyAdmin {
    votingStarted = true;
    votingEnded = false;
}

function endVoting() public onlyAdmin {
    votingEnded = true;
}

function vote(uint _candidateId) public {

    require(votingStarted == true, "Voting has not started");
    require(votingEnded == false, "Voting has ended");
    require(hasVoted[msg.sender] == false, "You have already voted");
    require(_candidateId > 0 && _candidateId <= candidatesCount, "Invalid candidate");

    hasVoted[msg.sender] = true;

    candidates[_candidateId].voteCount++;
}
```

```
    function getVotes(uint _candidateId) public view returns(uint) {
        return candidates[_candidateId].voteCount;
    }
}
```

Step 4: Compile the Smart Contract

Step 5: Deploy the Contract

Step 6: Start Voting

Click:

startVoting

Then click:

Transact

No parameter needed.

Voting is now ON.

Step 7: Vote for Candidate

Under `vote`:

You will see an input box.

Type:

1

Then click:

Transact

This means:

- Candidate 1 gets one vote

You can also vote:

- 2 for Bob
- 3 for Charlie

Step 8: Check Votes

Under:

getVotes

Type:

1

Then click:

Call

Output:

1

means candidate 1 has 1 vote.

Step 9: End Voting

Click:

endVoting

Then click:

Transact

Voting stops.

Step 10: Test One Vote Per Address To verify security:

step : Try Voting Again with SAME Account

Without changing account:

1. **Open** `vote`
2. **Enter:** 2
3. **Click:**

Transact

Now transaction will FAIL.

You will see error like:

You have already voted

This proves:
✅ Same address cannot vote twice.

2. Crowdfunding Platform Problem Statement: Create a smart contract that allows users to create fundraising campaigns and accept donations. If the funding goal is not met by the deadline, donors should be able to withdraw their funds.

[tell me one thing when creating campaign we give goal and duration only not id and set value is 0 .but when donate it ask for campaign id in text field and value set to 4
but how i will know what campaign id is because during creating campaign we don't give id =So every new campaign gets the next number automatically.]

Easy Smart Contract (Solidity):

```
// SPDX-License-Identifier: MIT

pragma solidity ^0.8.0;

contract Crowdfunding {

    struct Campaign {

        address payable creator;

        uint goal;

        uint deadline;
```

```
        uint amountRaised;
        bool fundsWithdrawn;
    }    uint public campaignCount;
mapping(uint => Campaign) public campaigns;
mapping(uint => mapping(address => uint)) public donations;
    // Create Campaign
    function createCampaign(uint _goal, uint _durationInMinutes) public {
        campaignCount++;
        campaigns[campaignCount] = Campaign({
            creator: payable(msg.sender),
            goal: _goal,
            deadline: block.timestamp + (_durationInMinutes * 1 minutes),
            amountRaised: 0,
            fundsWithdrawn: false
        });
    }
 // Donate to Campaign
    function donate(uint _campaignId) public payable {
        Campaign storage campaign = campaigns[_campaignId];
        require(block.timestamp < campaign.deadline, "Campaign ended");
        campaign.amountRaised += msg.value;
        donations[_campaignId][msg.sender] += msg.value;
    }
```

```
    // Creator Withdraws Funds
    function withdrawFunds(uint _campaignId) public {
        Campaign storage campaign = campaigns[_campaignId];
        require(msg.sender == campaign.creator, "Not creator");
require(block.timestamp >= campaign.deadline, "Campaign still active");
        require(campaign.amountRaised >= campaign.goal, "Goal not reached");
        require(!campaign.fundsWithdrawn, "Already withdrawn");

        campaign.fundsWithdrawn = true;

        (bool success, ) = campaign.creator.call{value: campaign.amountRaised}("");
require(success, "Transfer failed");

    }
    // Donor Refund
    function refund(uint _campaignId) public {
        Campaign storage campaign = campaigns[_campaignId];
        require(block.timestamp >= campaign.deadline, "Campaign still active");
        require(campaign.amountRaised < campaign.goal, "Goal was reached");
        uint donatedAmount = donations[_campaignId][msg.sender];
        require(donatedAmount > 0, "No donation found");
        donations[_campaignId][msg.sender] = 0;
  (bool success, ) = payable(msg.sender).call{value: donatedAmount}("");
```

```
require(success, "Refund failed");


    }
}
```

Step 2: Create Solidity File

Crowdfunding.sol

Step 3: Compile Contract

Step 4: Deploy Contract

Step 5: Create Campaign

In the input box enter:

5000000000000000000,

2

Enter : Transact

Step 6: Donate to Campaign

Step 7: Test Goal Success

If donations ≥ goal:

- creator calls:

  withdrawFunds(1)
- VALUE = 0

Unit:

wei

Funds transferred to creator.

Step 8: Test Refund

If donations < goal after deadline:

- donor calls:

  refund(1)

Donor receives refund.

3.Decentralized To-Do List Problem Statement: Develop a smart contract that allows users to add, update, and mark tasks as completed. Each user's tasks should be stored independently and be accessible only by the user.

Step 2: Create a New Solidity File

TodoList.sol

```
// SPDX-License-Identifier: MIT

pragma solidity ^0.8.0;

contract TodoList {
    struct Task {
        uint id;
        string content;
        bool completed;
    }
    mapping(address => Task[]) private userTasks;
    function addTask(string memory _content) public {
        uint taskId = userTasks[msg.sender].length;
        userTasks[msg.sender].push(
            Task(taskId, _content, false)
        );
```

```
}
function getTask(uint _id) public view returns (
    uint,
    string memory,
    bool
) {
    Task memory task = userTasks[msg.sender][_id];
    return (
        task.id,
        task.content,
        task.completed
    );
}
function updateTask(uint _id, string memory _newContent) public {
    userTasks[msg.sender][_id].content = _newContent;
}
function toggleCompleted(uint _id) public {
    userTasks[msg.sender][_id].completed =
        !userTasks[msg.sender][_id].completed;
}
function getTaskCount() public view returns (uint) {
    return userTasks[msg.sender].length;
}
```

}

Step 4: Compile the Contract

Step 5: Deploy the Contract

Step 6: Test addTask()

Under deployed contracts:

1. Find:

   addTask

2. Enter task:

   Buy groceries

3. Click transact

Task is added.

Step 7: Check Number of Tasks

1. **Click:**

   getTaskCount

Output:

1

Step 8: View Task

1. **In:**

   getTask

2. **Enter:**

   0

3. **Click call**

Output example:

0:

uint256: 0

1:

string: Buy groceries

2:

bool: false

Step 9: Update Task

1. In:

   updateTask

2. Enter:

   0

3. Enter new text:

   Buy groceries and milk

4. Click transact

Task updated successfully.

Step 10: Mark Task as Completed

1. In:

   toggleCompleted

2. Enter:

   0

3. Click transact

Now task becomes completed.

Check again using `getTask(0)`.

Output:

completed = true

Step 11: Verify Independent User Storage

To prove every user has separate tasks:

1. Change account from the account dropdown in Remix
2. Add another task
3. Call:

   getTaskCount

You will notice:

- New account has separate tasks

- Old account tasks are inaccessible

This happens because of:

mapping(address => Task[]) private userTasks;

Each Ethereum address has its own task list.

4.**Simple Bank (Wallet System) Problem Statement: Build a smart contract that functions as a simple bank. Users should be able to deposit, withdraw, and check their balance. Only the account holder should have access to their funds.**

SimpleBank.sol

```
// SPDX-License-Identifier: MIT

pragma solidity ^0.8.0;


contract SimpleBank {


    // Store balance of each user

    mapping(address => uint) private balances;

 // Deposit Ether into contract

    function deposit() public payable {

        balances[msg.sender] += msg.value;

    }

// Withdraw Ether from contract

    function withdraw(uint amount) public {


        // Check sufficient balance

        require(balances[msg.sender] >= amount, "Insufficient balance");
```

```
        // Deduct balance first
        balances[msg.sender] -= amount;
    // Transfer Ether
        payable(msg.sender).transfer(amount);
    }
 // Check own balance
    function checkBalance() public view returns(uint) {
        return balances[msg.sender];
    }
}
```

Step 6: Deposit Ether

To deposit money:

1. In Deploy section find:

   VALUE

2. Enter amount:

   1

3. Change unit from:

   wei → ether

4. Click:

   deposit

Now 1 Ether is stored in your wallet balance inside the contract.

Step 7: Check Balance

Click:

```
checkBalance
```

You will see balance in:

```
wei
```

Example:

```
1000000000000000000
```

This equals:

```
1 Ether
```

Step 8: Withdraw Ether

To withdraw:

1. In withdraw input box enter:

```
500000000000000000
```

This equals:

```
0.5 Ether
```

2. Click:

```
withdraw
```

Now remaining balance becomes:

```
0.5 Ether
```

Step 9: Verify Updated Balance

Click:

```
checkBalance
```

Expected output:

```
500000000000000000
```

Step 10: Test Multiple Users

To prove every user has separate funds:

1. Change account from ACCOUNT dropdown
2. Deposit Ether from another account
3. Call:

```
checkBalance
```

You will notice:

- Each account has separate balance
- One user cannot access another user's funds

This happens because of:

```solidity
mapping(address => uint) private balances;
```

Each Ethereum address gets its own wallet storage.

5.Supply Chain Management Problem Statement: Create a smart contract to track products from manufacturer to consumer. Each step in the supply chain should be recorded immutably with timestamps and participant addresses.

SupplyChain.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract SupplyChain {

    // Product structure
    struct Product {
        uint id;
        string name;
        string currentStage;
        address currentHolder;
        bool exists;
    }

    // Tracking history structure
    struct Tracking {
        string stage;
        address participant;
        uint timestamp;
    }

    // Store products
    mapping(uint => Product) public products;

    // Store tracking history
    mapping(uint => Tracking[]) public productHistory;

    // Add product
    function addProduct(
        uint _id,
        string memory _name
    ) public {

        require(!products[_id].exists, "Product already exists");

        products[_id] = Product({
            id: _id,
            name: _name,
            currentStage: "Manufactured",
```

```
        currentHolder: msg.sender,
        exists: true
    });

    productHistory[_id].push(
        Tracking(
            "Manufactured",
            msg.sender,
            block.timestamp
        )
    );
}

// Update supply chain stage
function updateStage(
    uint _id,
    string memory _newStage
) public {

    require(products[_id].exists, "Product does not exist");

    products[_id].currentStage = _newStage;
    products[_id].currentHolder = msg.sender;

    productHistory[_id].push(
        Tracking(
            _newStage,
            msg.sender,
            block.timestamp
        )
    );
}

// Get tracking count
function getTrackingCount(uint _id)
    public
    view
    returns(uint)
{
    return productHistory[_id].length;
}

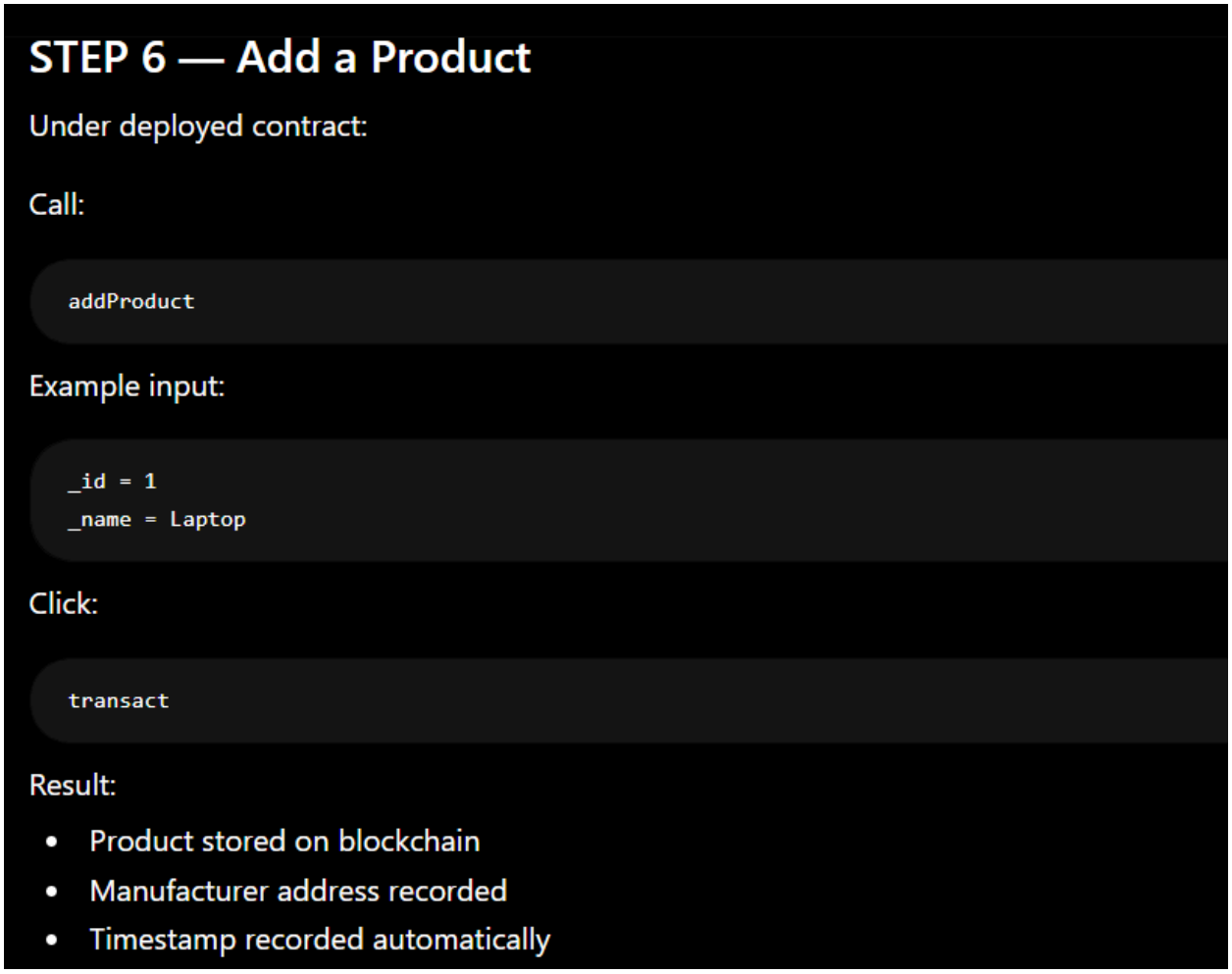
// Get tracking details
function getTracking(
```

```
        uint _id,
        uint _index
    )
        public
        view
        returns(
            string memory,
            address,
            uint
        )
    {
        Tracking memory t = productHistory[_id][_index];

        return (
            t.stage,
            t.participant,
            t.timestamp
        );
    }
}
```

Step 4: Compile the Contract

Step 5: Deploy the Contract



STEP 7 — Check Product Details

Call:

products(1)

You will see:

- **Product ID**
- **Product name**
- **Current stage**
- **Current holder address**
- **Exists = true**

Example:

1,
"Laptop",

"Manufactured",
0x123...,
True

STEP 8 — Update Supply Chain Stage

Now move product through supply chain.

Example stages:

- **Packed**
- **Shipped**
- **Warehouse**
- **Retailer**
- **Delivered**

Call:

updateStage

Example:

_id = 1
_newStage = Shipped

Click:

transact

Blockchain now stores:

- **New stage**
- **Participant address**
- **Timestamp**

STEP 9 — Simulate Different Participants

To show different participants:

In Remix:

- Change account from account dropdown

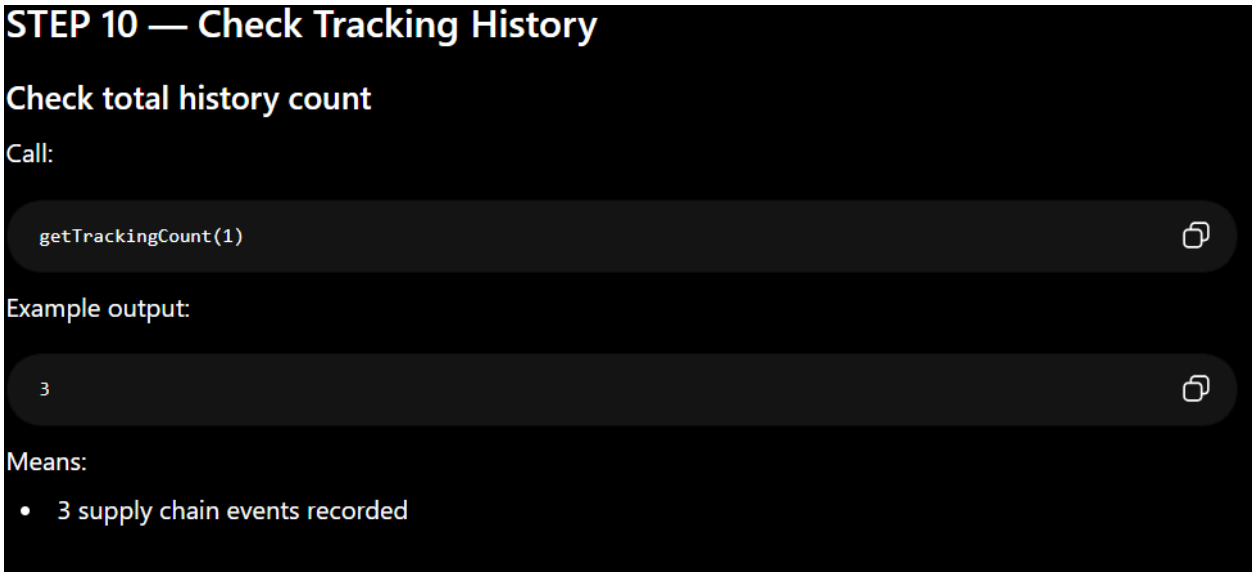
Example:

- Manufacturer account
- Distributor account
- Retailer account

Each account updates stage separately.

This records:

- Different Ethereum addresses
- Different timestamps



STEP 11 — View Individual Tracking Records

Call:

getTracking

Example:

_id = 1

_index = 0

Output:

"Manufactured"

0xManufacturerAddress

timestamp

Next:

_index = 1

Output:

"Shipped"

0xDistributorAddress

timestamp

STEP 12 — Important Concept Explanation

Immutable Storage

All records are stored on blockchain:

- Cannot be changed
- Cannot be deleted easily
- Fully transparent